



Como desenvolver aplicações web na arquitetura MVC

Instrutor: Johnny Souto
Aula Teste Senai – SENAI/SP

A dor da manutenção

"O cliente pediu só para trocar a cor do botão Excluir. Mas, para isso, você abre um arquivo onde também estão o SQL do banco e as regras do sistema. E se você quebrar algo sem querer?"



Manutenção

Todo o código em um só lugar: qualquer mudança mexe no todo.



Risco

Alterar a tela pode quebrar a regra de negócio.



Equipe

Duas pessoas no mesmo arquivo geram conflito.

Vamos ver como é esse código na prática →

O monólito sem organização: tudo misturado

■ Controller ■ Model ■ View

1 arquivo · 153 linhas

4 rotas · 5 consultas SQL · HTML e CSS

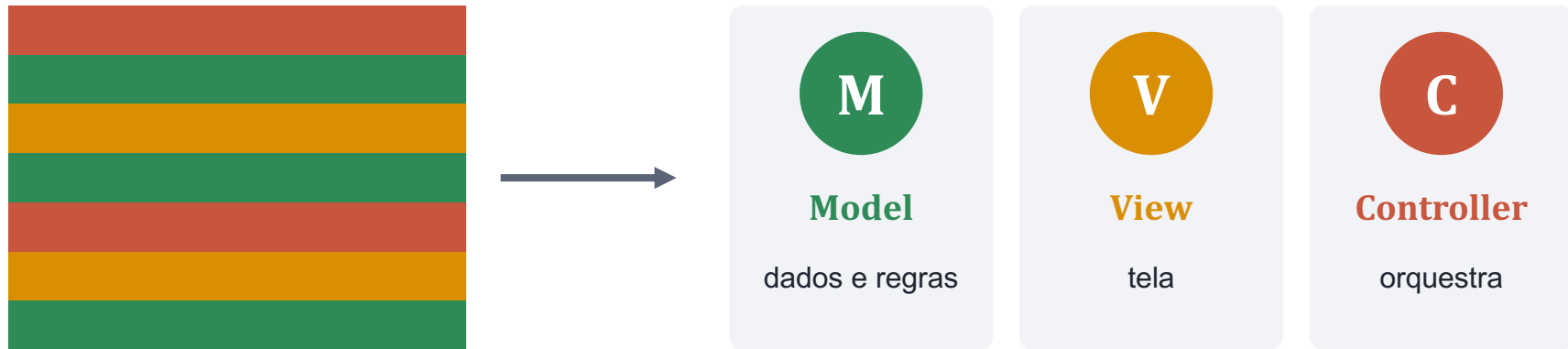
Mesmo sistema do app MVC:
mesmas funções, mesmo banco de
dados. Só muda a organização.

Situação: organizar o app de
anotações em MVC e criar a função
"Excluir nota".

```
// app-sem-mvc/server.js (trechos)
const pool = new Pool({ host: process.env.DB_HOST, ... });
app.get("/", async (requisicao, resposta) => {
  const resultado = await pool.query(
    "SELECT * FROM notas ORDER BY " +
    "favorita DESC, criado_em DESC");
  const linhas = resultado.rows.map((nota) => `
    <li><form action="/notas/${nota.id}/favorita">
      <strong>${escapar(nota.titulo)}</strong>
      <form action="/notas/${nota.id}/excluir">
    </li>`).join("");
  resposta.send(`<html><style>body { ... }</style>
    <form method="POST" action="/notas">...</form>
    <ul>${linhas}</ul></html>`);
});
app.post("/notas", async (requisicao, resposta) => {
  const { titulo, conteudo } = requisicao.body;
  if (!titulo) return resposta.redirect("/?erro=...");
  await pool.query("INSERT INTO notas ...", [titulo]);
  resposta.redirect("/");
});
// ... + rotas de excluir e favoritar, tudo igual
```

A solução: arquitetura MVC

Para resolver essa dor, a engenharia de software usa padrões de arquitetura. O MVC separa a aplicação em três responsabilidades.



1 arquivo, tudo misturado

3 camadas, cada uma muda por um único motivo

- Criado por Trygve Reenskaug (Xerox PARC, 1979) para interfaces gráficas
- Adaptado para a web por frameworks como Spring, ASP.NET, Rails e Laravel

Quem faz o quê



Model

Onde ficam os dados
e as regras?

`models/Nota.js`



View

O que o usuário vê?

`public/ (HTML, CSS, JS)`



Controller

O que fazer quando o
usuário age?

`controllers/
notasController.js`



Model

`src/models/Nota.js`

Representa os dados e as regras de negócio da aplicação.

O que faz

- Define como é uma "nota"
- Valida regras (título obrigatório)
- Salva, busca e remove no banco

O que NÃO faz

- Não monta tela nem HTML
- Não sabe qual botão foi clicado

No restaurante: é a cozinha — conhece as receitas e os ingredientes, e prepara o prato.



`public/index.html · script.js`

É a camada de apresentação: tudo o que o usuário vê e com que interage.

O que faz

- Exibe os dados (lista de notas)
- Mostra formulários e botões
- Envia a ação do usuário

O que NÃO faz

- Não contém regra de negócio
- Não acessa o banco de dados

No restaurante: é o prato servido e o cardápio — é o que o cliente vê.



Controller

`src/controllers/notasController.js`

É o intermediário: recebe a requisição, aciona o Model e devolve a resposta.

O que faz

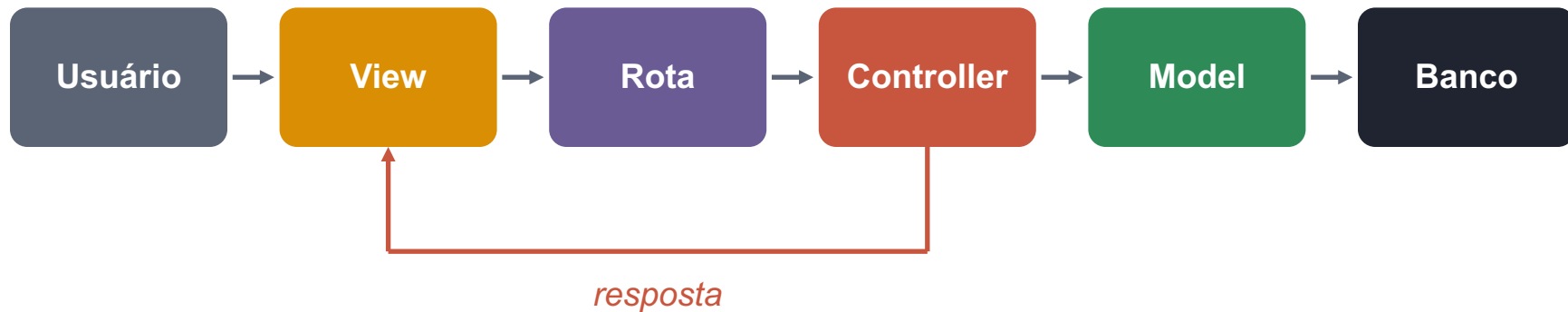
- Recebe a requisição (via rota)
- Chama o Model certo
- Decide a resposta (status, dados)

O que NÃO faz

- Não guarda regra de negócio
- Não monta o visual da tela

No restaurante: é o garçom — anota o pedido, leva à cozinha e traz o prato.

O caminho de um clique



Cada camada conversa só com a vizinha.

Por que usar MVC?

1

Manutenção

Mudou o visual? Mexe só na View. Mudou a regra? Só no Model.

2

Testes

Cada camada pode ser testada separadamente.

3

Equipe

Front-end e back-end evoluem em paralelo, sem conflito.

Usado em Spring, ASP.NET, Laravel, Ruby on Rails, Django e Express.

5 passos para criar qualquer funcionalidade

1

Model

O que fazer com os dados?

2

Controller

Como receber o pedido e responder?

3

Rota

Qual URL chama o Controller?

4

View

Como o usuário aciona e vê o resultado?

5

Testar

Funciona de ponta a ponta?

Agora, ao vivo: a funcionalidade "Excluir nota".

Organização das pastas

```
app-mvc/  
├── db/init.sql  
├── src/  
│   ├── config/db.js  
│   ├── models/Nota.js  
│   ├── controllers/notasController.js  
│   ├── routes/notasRoutes.js  
│   └── app.js  
└── public/
```

MODEL

models/

CONTROLLER

controllers/

ROTAS

routes/

VIEW

public/

Passo 1 · Model

src/models/Nota.js

```
static async remover(id) {  
  if (!Number.isInteger(Number(id))) {  
    throw new Error("Id inválido");  
  }  
  
  await pool.query(  
    "DELETE FROM notas WHERE id = $1",  
    [id]  
  );  
}
```

Regra: o id precisa ser um número.

Depois, apaga do banco.

Não sabe nada de botão ou tela.

Passo 2 · Controller

src/controllers/notasController.js

```
async function remover(requisicao, resposta) {  
  try {  
    await Nota.remover(requisicao.params.id);  
    resposta.status(204).send();  
  } catch (erro) {  
    resposta.status(400)  
      .json({ erro: erro.message });  
  }  
}
```

Pega o id da URL,
chama o Model e
responde.

204 = deu certo.
400 = erro tratado, o
servidor não cai.

Não acessa o banco.

Passo 3 · Rota

src/routes/notasRoutes.js

```
router.get("/notas", controller.listar);  
router.post("/notas", controller.criar);  
router.delete("/notas/:id",  
              controller.remover);
```

A porta de entrada.

GET → lista

POST → cria

DELETE → remove

Passo 4 · View

public/script.js – dentro do notas.forEach

```
const botao = document.createElement("button");  
botao.textContent = "Excluir";  
botao.onclick = async () => {  
  await fetch(`/api/notas/${nota.id}`,  
    { method: "DELETE" });  
  carregarNotas();  
};  
item.appendChild(botao);
```

Só cria o botão e
envia o pedido.

Nenhuma regra aqui.

Botão vermelho?
Mexe só na View.

Testar de ponta a ponta

- ✓ Criar uma nota → aparece na lista
- ✓ Clicar em "Excluir" → some da lista
- ✓ Recarregar a página → continua fora (saiu do banco)

Pergunta: e se a regra fosse "só excluir notas com mais de 1 dia"? Em qual camada?

Hora do Quiz!

kahoot.it

PIN: _____

5 perguntas · 20 segundos cada

O que aprendemos hoje

Model

dados e regras

View

tela

Controller

orquestra

Receita: Model → Controller → Rota → View → Testar

Vantagens: manutenção fácil, testes isolados, equipe trabalhando em paralelo

Desafio para casa: desenvolva a função "Editar nota" seguindo a mesma receita — Model → Controller → Rota → View → Testar.

Para estudar mais

- GAMMA et al. Padrões de Projeto. Bookman, 2000.
- FOWLER, M. Padrões de Arquitetura de Aplicações Corporativas. Bookman, 2006.
- SOMMERVILLE, I. Engenharia de Software. 10. ed. Pearson, 2018.
- MDN Web Docs — MVC: developer.mozilla.org/pt-BR/docs/Glossary/MVC
- Express.js — Roteamento: expressjs.com/pt-br/guide/routing.html

Obrigado!